第 17 天 • 11/18 • 今日知识点回顾

apply/map/lambda + 多级索引：什么时候该用、什么时候别用

目标：判断一段代码「该向量化还是该 apply」，并能用多级索引取出任意切片。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 判断一段代码「该向量化还是该 apply」，并能用多级索引取出任意切片。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 28 分钟（边看边在本机敲一遍）

P30	Pandas2高级函数应用	约 10 分钟
P29	Pandas2数据迭代操作	约 6 分钟
P84	函数进阶之函数作为参数传递	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch3.rst`

第三章 索引：多层索引的 loc / xs / IndexSlice 三种取法

知识点 1/7 • apply 是「最后手段」，先问能不能向量化

- 一句话：能用 .str / .dt / 算术 / where 解决的，绝不用 apply。
- 代码：

```
import pandas as pd, numpy as np, pathlib, time
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
# 慢：apply + 自定义函数
t0 = time.perf_counter()
a = df['amount'].apply(lambda x: x * 0.9 if pd.notna(x) else x)
t1 = time.perf_counter()
# 快：向量化
b = df['amount'].where(df['amount'].isna(), df['amount'] * 0.9)
t2 = time.perf_counter()
print(f'apply {t1-t0:.4f}s  where {t2-t1:.4f}s  结果相同 {a.equals(b)}')
```

这一页只讲「apply 是「最后手段」，先问能不能向量化」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • apply 是「最后手段」，先问能不能向量化（续）

★ 1163 行时 apply 可能只慢 3–5 倍，看着无所谓；到 100 万行时是 50 倍以上。

- apply 本质是 Python 循环，向量化走的是 C。

知识点 2/7 • map：按字典做值替换，比 replace 更可控

- 一句话：s.map({'M': '男', 'F': '女'}) 找不到的变 nan —— 这一点很关键。
- 代码：

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
print(u['gender'].value_counts(dropna=False).to_dict())
m = {'M': '男', 'F': '女', '男': '男', '女': '女'}
u['g2'] = u['gender'].map(m)
print(u['g2'].value_counts(dropna=False).to_dict())
```

★ 本机实测：gender 有六种写法（男/女/M/F/未知/空），map 之后「未知」和空变成 nan。想保留成「未知」就写 m.get(x, '未知') 用 apply，或者先在字典里补 {'未知': '未知'}。map 的「找不到→nan」是隐式行为，最容易悄悄丢掉一类数据。

这一页只讲「map：按字典做值替换，比 replace 更可控」。讲完就翻页，下一页是另一个知识点。

知识点 3/7 • apply 的正确用法：一行里要看好几列

- 一句话：df.apply(func, axis=1) 把每一行当 Series 传给函数 —— 这是向量化做不到的场景。
- 代码：

知识点 3/7 • apply 的正确用法：一行里要看好几列（续）

```
def level(r):
if pd.isna(r['amount']):
return '金额缺失'
if r['amount'] >= 2000:
return '高客单'
if r['amount'] >= 500:
return '中客单'
return '低客单'
df['level'] = df.apply(level, axis=1)
print(df['level'].value_counts().to_dict())
```

★ axis=1 比 axis=0 慢得多（逐行构造 Series）。数据超过 10 万行时，改用 np.select 或 pd.cut —— 下面两条就是替代方案。

知识点 4/7 • 两个向量化替代：np.select 与 pd.cut

- 一句话：多条件分支用 np.select；等距分箱用 pd.cut；分位分箱用 pd.qcut。
- 代码：

```
cond = [df['amount'].isna(), df['amount'] >= 2000, df['amount'] >= 500]
df['level2'] = np.select(cond, ['金额缺失', '高客单', '中客单'], default='低客单')
print((df['level'] == df['level2']).mean())          # 1.0 完全一致
df['bin'] = pd.cut(df['amount'], bins=[0, 500, 2000, np.inf],
labels=['低', '中', '高'], right=False)
print(df['bin'].value_counts(dropna=False).to_dict())
```

★ np.select 的条件是「从上往下第一个为 True 的生效」，顺序错了结果就错 ——和 if/elif 一样的规则。pd.cut 的 right=False 表示左闭右开 [0,500)，默认 right=True 是 (0,500]，边界值会掉进不同的桶。

知识点 5/7 • 多级索引：groupby 两个键之后发生了什么

- 一句话：groupby(['city','category']) 得到一个 MultiIndex，行标签是元组。
- 代码：

```
paid = df[df['status'] == 'paid']
g = paid.groupby(['city', 'category'])['amount'].sum()
print(type(g.index))                    # MultiIndex
print(g.index.names)                    # ['city', 'category']
print(g.head(3).to_string())
print(g.loc[('北京', '数码')])          # 按元组取单个值
```

★ 单个键用 g.loc['北京'] 也能取，但会返回一个 Series（4 个品类）。

- 要取「北京 + 数码」必须写成元组 ('北京', '数码')。

知识点 6/7 • xs 与 IndexSlice：取某一层的切片

- 一句话：g.xs('数码', level='category') 取所有城市在数码上的值。
- 代码：

```
print(g.xs('数码', level='category').sort_values(ascending=False).to_string())
print(g.loc[(slice(None), '数码'), ].sort_values(ascending=False).head(3).to_string())
idx = pd.IndexSlice
print(g.loc[idx[:, '数码']].head(3).to_string())
```

★ 三种写法等价，但 xs 最短、最不容易写错。IndexSlice 适合「同时切两层」的复杂场景（比如 loc[idx['北京':'上海', '数码':'家居']]）。

知识点 7/7 • reset_index / unstack / stack：三层索引的三个出口

- 一句话：reset_index 变平表、unstack 变宽表、stack 变长表。
- 代码：

```
print(g.reset_index().head(3).to_string(index=False))   # 变回普通列
print(g.unstack(fill_value=0).shape)                    # (9, 4) 宽表
print(g.unstack().stack().shape)                        # 又变回来（nan 会被丢）
```

- （stack() 默认 dropna=True，会**丢掉** nan 的组合 ——unstack().stack() 不一定还原成原样。要严格还原写 stack(dropna=False)。）

代码速查：今天这几段，要在本机跑通（1/7）

写 apply_practice.py，同时验证「两种写法结果一致」：

```
import pandas as pd, numpy as np, pathlib, time
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

1) apply vs 向量化 计时

t0 = time.perf_counter()

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/7）

```
a = df['amount'].apply(lambda x: round(x * 0.9, 2) if pd.notna(x) else x)
```

t1 = time.perf_counter()

b = (df['amount'] * 0.9).round(2)

t2 = time.perf_counter()

```
print(f'apply {t1-t0:.4f}s  vector {t2-t1:.4f}s  equal={a.equals(b)}')
```

```
# 2) 分档：apply / np.select / pd.cut 三种写法对齐
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/7）

```
def level(r):
```

```
    if pd.isna(r['amount']): return '缺失'
    if r['amount'] >= 2000: return '高'
    if r['amount'] >= 500: return '中'
    return '低'
```

```
df['lv_apply'] = df.apply(level, axis=1)
```

```
df['lv_select'] = np.select(
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/7）

```
    [df['amount'].isna(), df['amount'] >= 2000, df['amount'] >= 500],
```

['缺失', '高', '中'], default='低')

```
print('apply==select: ', (df['lv_apply'] == df['lv_select']).mean())
```

```
print(df['lv_select'].value_counts().to_dict())
```

3) map 归并 gender

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
```

代码速查：今天这几段，要在本机跑通（5/7）

```
m = {'M': '男', 'F': '女', '男': '男', '女': '女', '未知': '未知'}
```

u['g2'] = u['gender'].map(m)

```
print('map 后分布: ', u['g2'].value_counts(dropna=False).to_dict())
```

```
print('map 丢掉的行: ', u['g2'].isna().sum())
```

4) 多级索引三种取法

paid = df[df['status'] == 'paid']

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/7）

```
g = paid.groupby(['city', 'category'])['amount'].sum()
```

```
print(g.xs('数码', level='category').sort_values(ascending=False).head(3).to_string())
```

```
print(g.reset_index().shape, g.unstack(fill_value=0).shape)
```

验收（做到才算过关）：

① 第 1 题 equal=True，并报出两种写法的耗时比；

② 第 2 题 apply 与 np.select 结果 100% 一致；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/7）

③ 第 3 题能报出 map 之后有多少行变 nan（就是「未知」+ 空的那些）。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ np.select 的条件是「从上往下第一个为 True 的生效」，顺序错了结果就错 ——和 if/elif 一样的规则。pd.cut 的 right=False 表示左闭右开 [0,500)，默认 right=True 是 (0,500]，边界值会掉进不同的桶。

★ 三种写法等价，但 xs 最短、最不容易写错。IndexSlice 适合「同时切两层」的复杂场景（比如 loc[idx['北京':'上海', '数码':'家居']]）。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：1163 行时 apply 可能只慢 3–5 倍，看着无所谓；到 100 万行时是 50 倍以上。
- [ ] 我能不看资料写出：单个键用 g.loc['北京'] 也能取，但会返回一个 Series（4 个品类）。
- [ ] 我能不看资料说出：要取「北京 + 数码」必须写成元组 ('北京', '数码')。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 18 天 • 可视化：matplotlib 中文设置 + 三张有业务含义的图

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-18-第17天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data